Infrastructure Orchestration & Automation

Bloc 3 — Implémenter et automatiser le déploiement des solutions de données

Rapport de TP — Jour 2

Containeriser avec Docker, déployer avec Kubernetes

Étudiante	Bouchera MAROUF
Formation	Master Data Engineering & Intelligence Artificielle
Enseignant	Dr. Abdelhak TOUITI
Compétences visées	C7.1, C8.2
Outils	Docker, Docker Compose, Kubernetes (Minikube)
Date	29 septembre 2026

Sommaire

1. Introduction et environnement de travail

2. Préparation de l'API

3. Écriture du Dockerfile

4. Assemblage API + base de données avec Docker Compose

5. Vérification de la persistance des données

6. Installation et démarrage du cluster Minikube

7. Déploiement de l'API sur Kubernetes

8. Résilience (auto-réparation) et montée en charge

9. Versionnement du projet

10. Réponses aux questions de compréhension

11. Difficultés rencontrées et remarques

12. Conclusion

1. Introduction et environnement de travail

L'objectif de ce TP est de containeriser un petit service de données — une API Python (Flask) qui lit une table de ventes dans PostgreSQL —, de le faire fonctionner localement avec Docker Compose, puis de le déployer sur un cluster Kubernetes local (Minikube) afin d'observer deux propriétés clés d'un orchestrateur : l'**auto-réparation** et la **montée en charge**.

Architecture mise en place

Composant	Rôle	Où il tourne
API Flask *mon-service:1.0*	Expose /health et /ventes sur le port 8000	Docker Compose puis Kubernetes (3 à 5 pods)
PostgreSQL 16	Stocke la table *ventes*	Docker Compose (volume *donnees_db*)
Service *api-service*	Point d'accès stable et répartition de charge vers les pods	Kubernetes (NodePort)

Environnement technique

Élément	Version
Système	Windows 10 Pro 22H2
Docker Engine (Docker Desktop)	28.5.1
Docker Compose	v2.40.2
Minikube (driver docker)	v1.39.0
Kubernetes (cluster Minikube)	v1.37.0
kubectl (client)	v1.34.1
Image de base de l'API	python:3.12-slim

Arborescence finale du projet :

```
tp-docker-k8s/
├── api/
│   ├── app.py            # API Flask
│   ├── requirements.txt  # dépendances Python
│   ├── init.sql          # création + données de la table ventes
│   └── Dockerfile        # recette de l'image mon-service:1.0
├── docker-compose.yml    # API + PostgreSQL + volume
├── deployment.yaml       # Deployment Kubernetes (3 réplicas)
├── service.yaml          # Service Kubernetes (NodePort)
├── REPONSES.md
└── captures/             # captures d'écran du TP
```

2. Préparation de l'API (Étape 1)

Le dossier *tp-docker-k8s/api/* contient trois fichiers. Le fichier **app.py** définit deux routes : */health*, qui sert de sonde de santé et ne dépend de rien, et */ventes*, qui interroge la base. Les paramètres de connexion sont lus dans des **variables d'environnement** (avec des valeurs par défaut) : la même image peut ainsi être configurée différemment sous Compose ou sous Kubernetes sans être reconstruite.

api/app.py

```
import os
from flask import Flask, jsonify
import psycopg2

app = Flask(__name__)

def connexion_db():
    return psycopg2.connect(
        host=os.environ.get("DB_HOST", "db"),
        dbname=os.environ.get("DB_NAME", "ventes"),
        user=os.environ.get("DB_USER", "postgres"),
        password=os.environ.get("DB_PASSWORD", "postgres"),
    )

@app.route("/health")
def health():
    return jsonify(status="ok")

@app.route("/ventes")
def ventes():
    conn = connexion_db()
    cur = conn.cursor()
    cur.execute("SELECT id, produit, montant FROM ventes;")
    rows = cur.fetchall()
    conn.close()
    return jsonify(rows)

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8000)
```

Le fichier **requirements.txt** fige les versions des dépendances pour garantir des constructions reproductibles, et **init.sql** crée la table de démonstration avec deux lignes.

api/requirements.txt

```
flask==3.0.3
psycopg2-binary==2.9.9
```

api/init.sql

```
CREATE TABLE ventes (
    id SERIAL PRIMARY KEY,
    produit VARCHAR(50),
    montant NUMERIC
);
INSERT INTO ventes (produit, montant) VALUES
    ('Abonnement A', 120.00),
    ('Abonnement B', 89.90);
```

3. Écriture du Dockerfile (Étape 2)

api/Dockerfile

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
CMD ["python", "app.py"]
```

Explication des instructions :

- **FROM python:3.12-slim** : image de base légère contenant l'interpréteur Python.
- **WORKDIR /app** : répertoire de travail dans le conteneur.
- **COPY requirements.txt** puis **RUN pip install** : les dépendances sont installées *avant* la copie du code. Grâce au cache des couches Docker, une modification de *app.py* ne relance pas l'installation des paquets.
- **--no-cache-dir** : évite de conserver le cache pip dans l'image (image plus petite).
- **COPY . .** : copie du code source.
- **CMD** : commande exécutée au démarrage du conteneur.

L'image est construite avec *docker build -t mon-service:1.0 .* depuis le dossier *api/*.

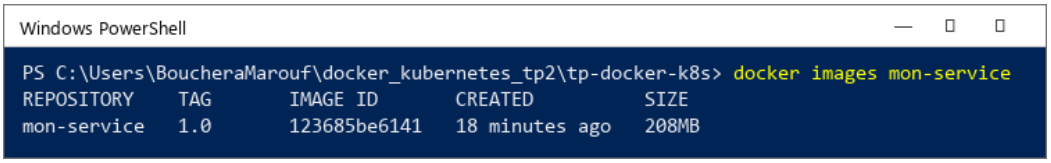


Figure 1 — l'image mon-service:1.0 apparaît bien dans la liste des images locales

4. Assemblage API + base de données avec Docker Compose (Étape 3)

docker-compose.yml

```
services:
  api:
    build: ./api
    ports:
      - "8000:8000"
    environment:
      DB_HOST: db
      DB_NAME: ventes
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: ventes
      POSTGRES_PASSWORD: postgres
    volumes:
      - donnees_db:/var/lib/postgresql/data
      - ./api/init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  donnees_db:
```

- Compose crée un **réseau commun** aux deux services avec une résolution DNS par nom : l'API joint la base simplement via l'hôte **db** (variable *DB_HOST: db*).
- **ports « 8000:8000 »** publie le port de l'API sur la machine hôte.
- **depends_on** démarre la base avant l'API (ordre de démarrage uniquement, pas d'attente que la base soit prête).
- Le volume nommé **donnees_db** est monté sur le répertoire de données de PostgreSQL.
- **init.sql** est monté dans */docker-entrypoint-initdb.d/* : l'image officielle PostgreSQL l'exécute automatiquement lors de la première initialisation (c'est-à-dire quand le volume est vide).

La pile est lancée avec *docker compose up -d*. Les deux conteneurs sont démarrés, */health* répond *ok* et */ventes* renvoie les deux lignes insérées par *init.sql*. Sous PowerShell, *Invoke-RestMethod* est utilisé plutôt que *curl* (alias d'*Invoke-WebRequest*) pour obtenir directement le JSON désérialisé.

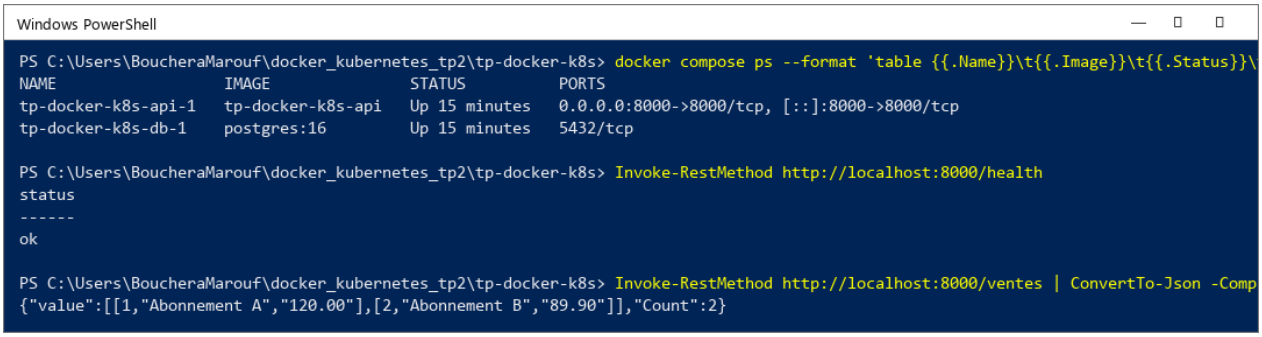


Figure 2 — conteneurs démarrés et réponses de l'API sur /health et /ventes

5. Vérification de la persistance des données (Étape 4)

Test 1 — arrêt / redémarrage. Après *docker compose down* (qui supprime les conteneurs) puis *docker compose up -d*, les données sont toujours présentes.

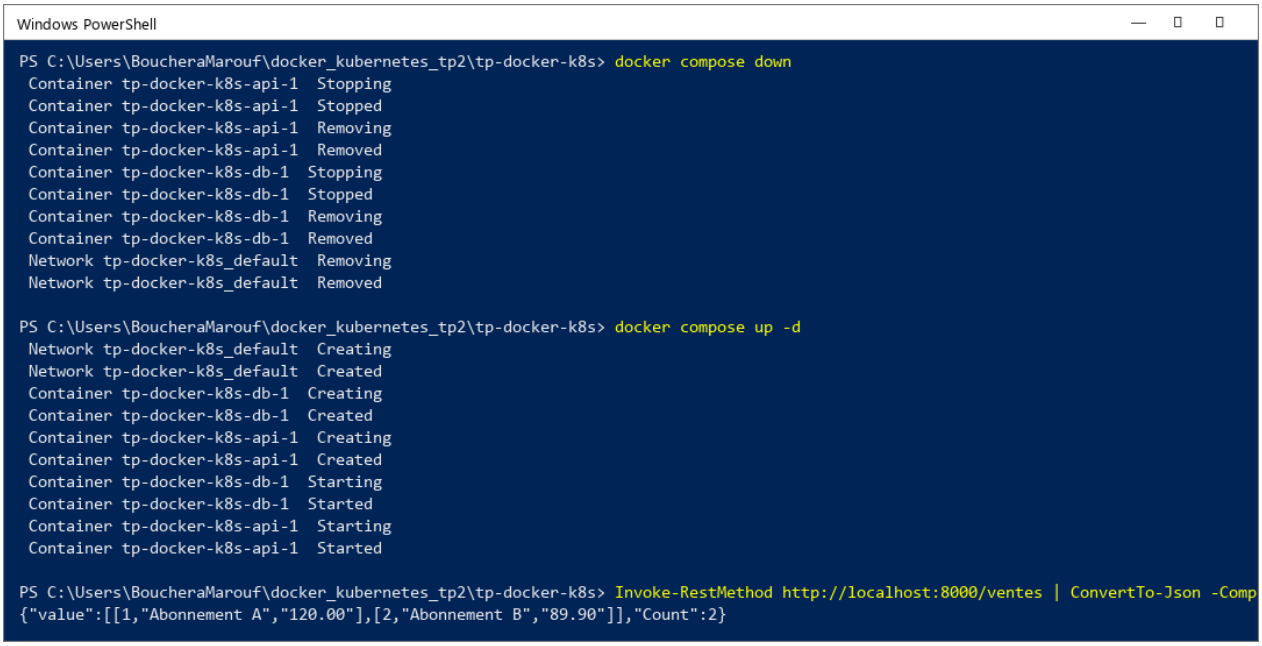


Figure 3 — les données sont toujours présentes après down / up

Test 2 — ajout d'une ligne. Pour rendre la preuve plus convaincante (les deux lignes initiales pourraient venir d'une nouvelle exécution d'*init.sql*), une troisième ligne « Test persistance » est ajoutée directement dans la base, puis la pile est de nouveau arrêtée et relancée : la ligne ajoutée a bien survécu. *init.sql* n'a pas été rejoué, car le volume n'était pas vide.

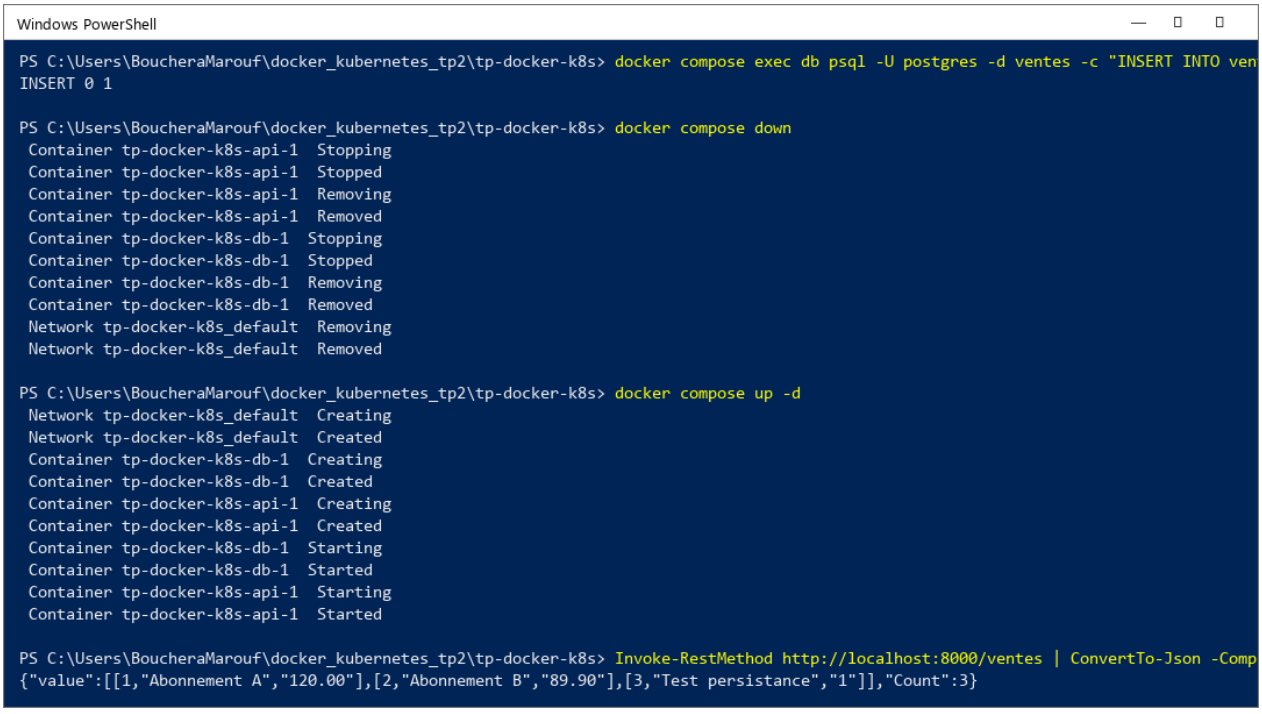


Figure 4 — une ligne ajoutée manuellement survit à la suppression des conteneurs

Test 3 — suppression du volume. Avec *docker compose down -v*, le volume est supprimé. Au redémarrage, PostgreSQL repart d'un répertoire de données vide, rejoue *init.sql*, et la ligne ajoutée a disparu.

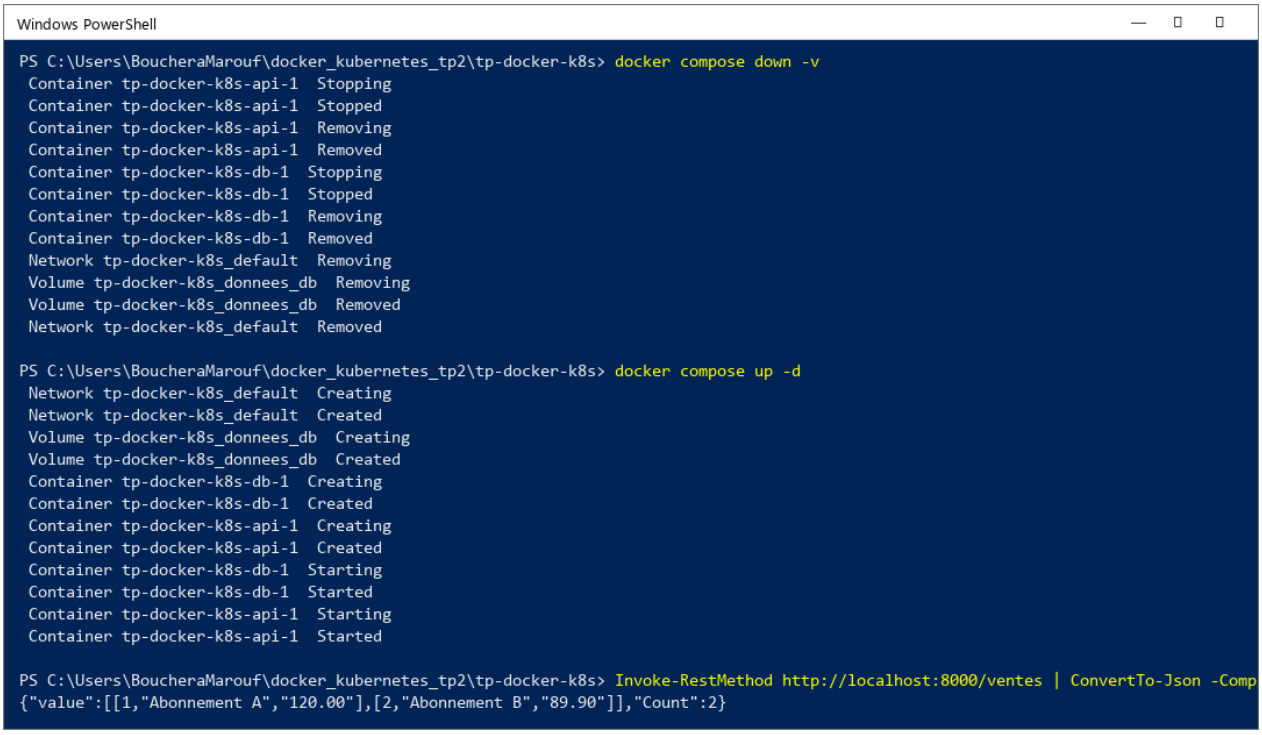


Figure 5 — après down -v, la ligne ajoutée est perdue : seules les données d'init.sql reviennent

Conclusion : c'est bien le **volume**, et non le conteneur, qui assure la persistance des données.

6. Installation et démarrage du cluster Minikube (Étape 5)

kubectl était déjà disponible (fourni par Docker Desktop). Minikube a été installé avec *winget install --id Kubernetes.minikube*, puis le cluster a été démarré avec *minikube start --driver=docker*. Avec ce driver, le nœud Kubernetes tourne lui-même dans un conteneur Docker : il n'est pas nécessaire d'activer Hyper-V ni d'installer VirtualBox. Le premier démarrage a téléchargé l'image du nœud (environ 500 Mo).

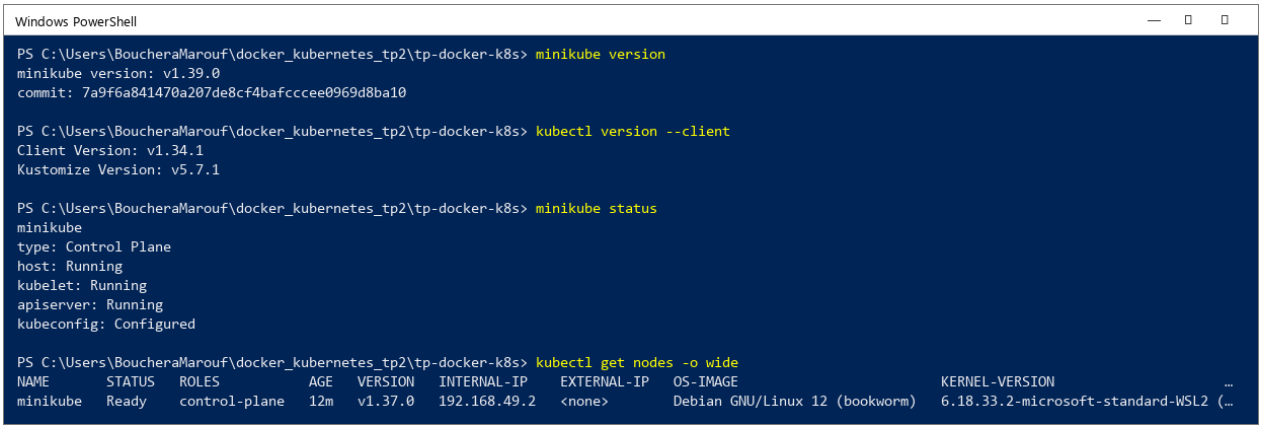


Figure 6 — versions des outils, état du cluster et nœud minikube au statut Ready

7. Déploiement de l'API sur Kubernetes (Étape 6)

6.1 — Rendre l'image disponible. Le cluster Minikube possède son propre magasin d'images, distinct de celui de Docker Desktop. La commande *minikube image load mon-service:1.0* y copie l'image locale. Comme le tag n'est pas *latest*, la politique par défaut est *IfNotPresent* : Kubernetes utilise l'image locale au lieu de chercher à la télécharger depuis Docker Hub.

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: mon-service
  template:
    metadata:
      labels:
        app: mon-service
    spec:
      containers:
        - name: api
          image: mon-service:1.0
          ports:
            - containerPort: 8000
          env:
            - name: DB_HOST
              value: db
```

- **replicas: 3** : l'état désiré est de 3 pods identiques.
- **selector.matchLabels** / **template.metadata.labels** : le Deployment reconnaît « ses » pods grâce au label *app: mon-service*.
- **template** : le modèle de pod (image, port, variables d'environnement) utilisé pour créer chaque réplica.

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: api-service
spec:
  selector:
    app: mon-service
  ports:
    - port: 8000
      targetPort: 8000
  type: NodePort
```

- Le Service sélectionne tous les pods portant le label *app: mon-service* et répartit le trafic entre eux.
- **port** est le port du Service, **targetPort** le port du conteneur.
- **type: NodePort** expose en plus le Service sur un port du nœud (plage 30000-32767), accessible depuis l'extérieur du cluster.

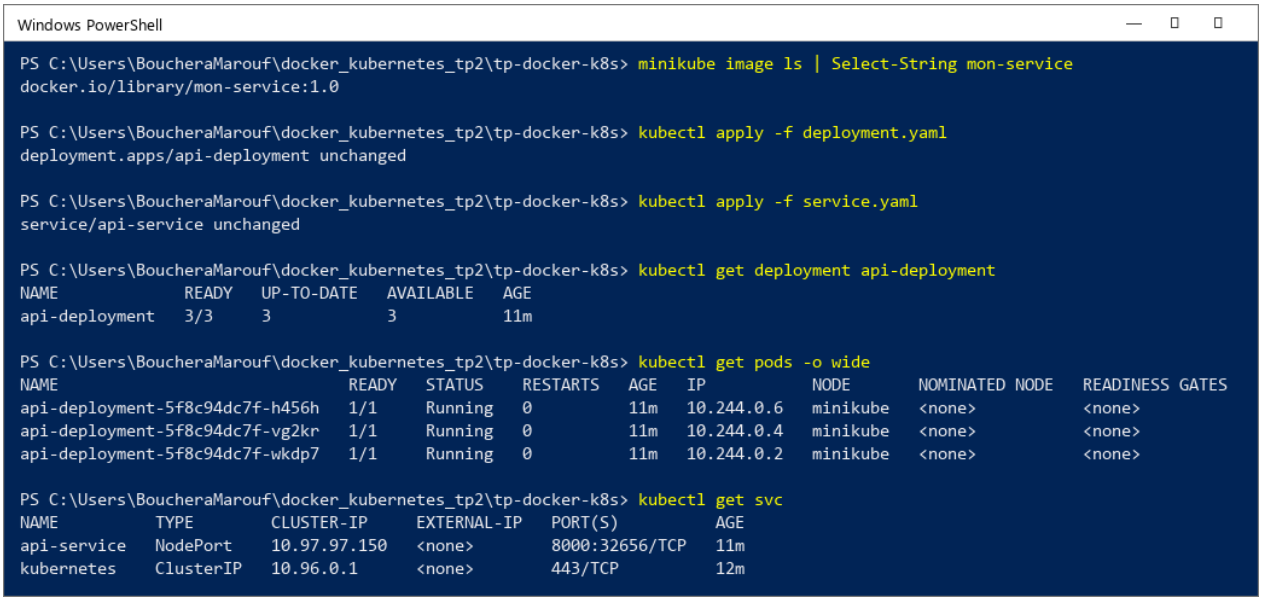


Figure 7 — image chargée, manifests appliqués : 3 pods Running et le service api-service

Test d'accès via le Service. Avec le driver docker sous Windows, l'IP du nœud n'est pas directement joignable depuis l'hôte : un *kubectl port-forward* vers le Service a été utilisé. La route */health* répond correctement. La route */ventes* n'aboutit pas, ce qui est **attendu** : comme le prévoit l'énoncé, la base de données n'a pas été déployée sur Kubernetes, et le nom *db* n'y est donc pas résolu.

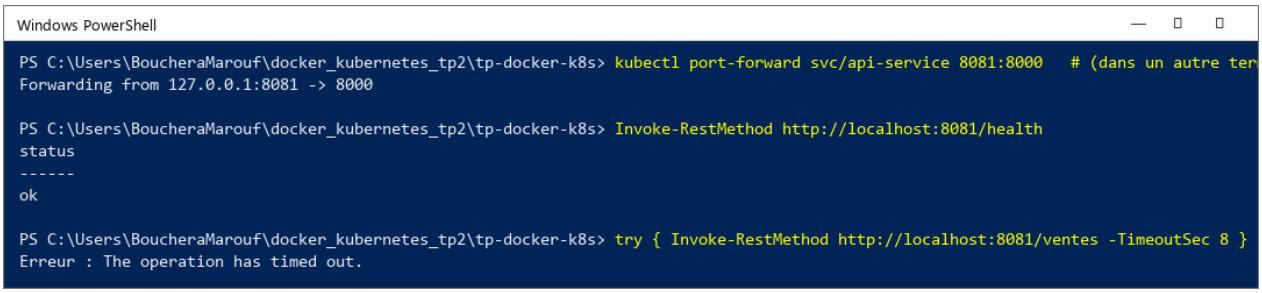


Figure 8 — l'API répond via le Service ; /ventes échoue faute de base dans le cluster (comportement attendu)

8. Résilience et montée en charge (Étape 7)

7.1 — Auto-réparation

Un des trois pods est supprimé manuellement. Juste après la suppression, le pod supprimé est en cours d'arrêt (*Terminating*), tandis qu'un **nouveau pod** (nom différent, âge de quelques secondes) est déjà créé pour le remplacer. Quelques secondes plus tard, le Deployment compte de nouveau **3 pods Running**.

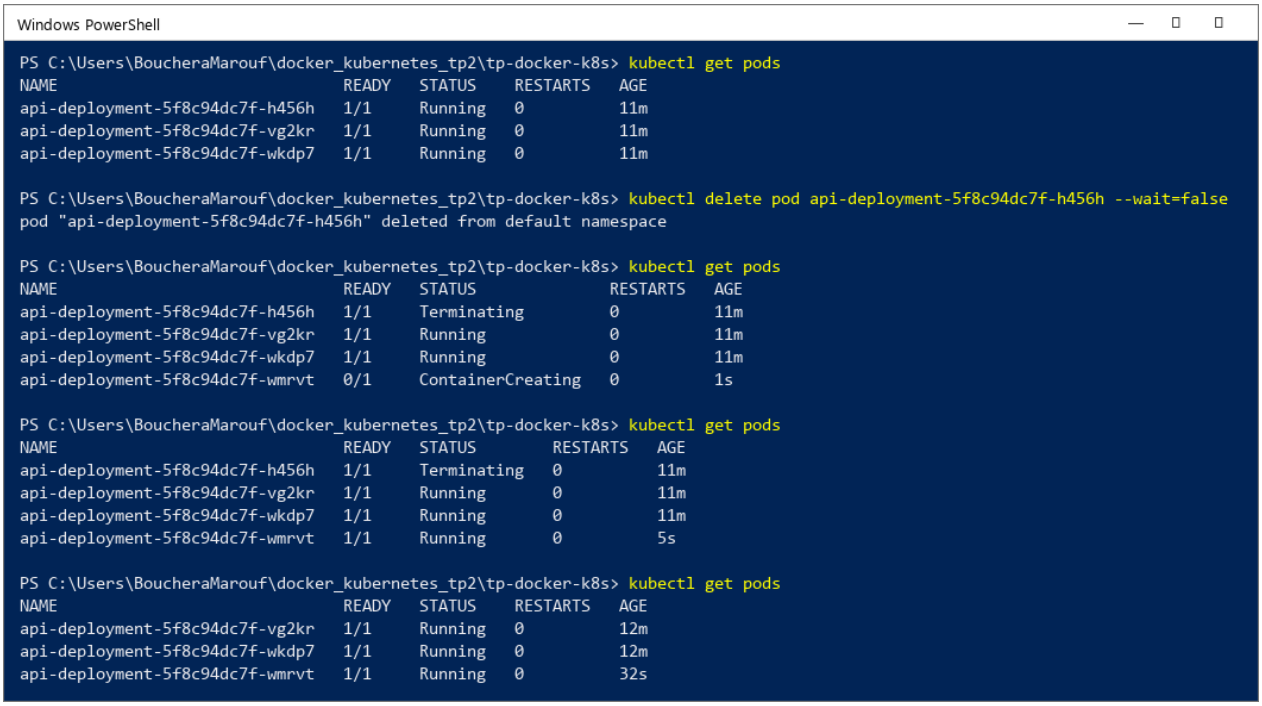


Figure 9 — suppression d'un pod et recréation automatique par Kubernetes

Le ReplicaSet (créé par le Deployment) compare en permanence le nombre de pods réels au nombre désiré. Dès qu'un pod disparaît, l'écart est détecté et corrigé. Les événements du ReplicaSet montrent bien la création des pods (SuccessfulCreate) :

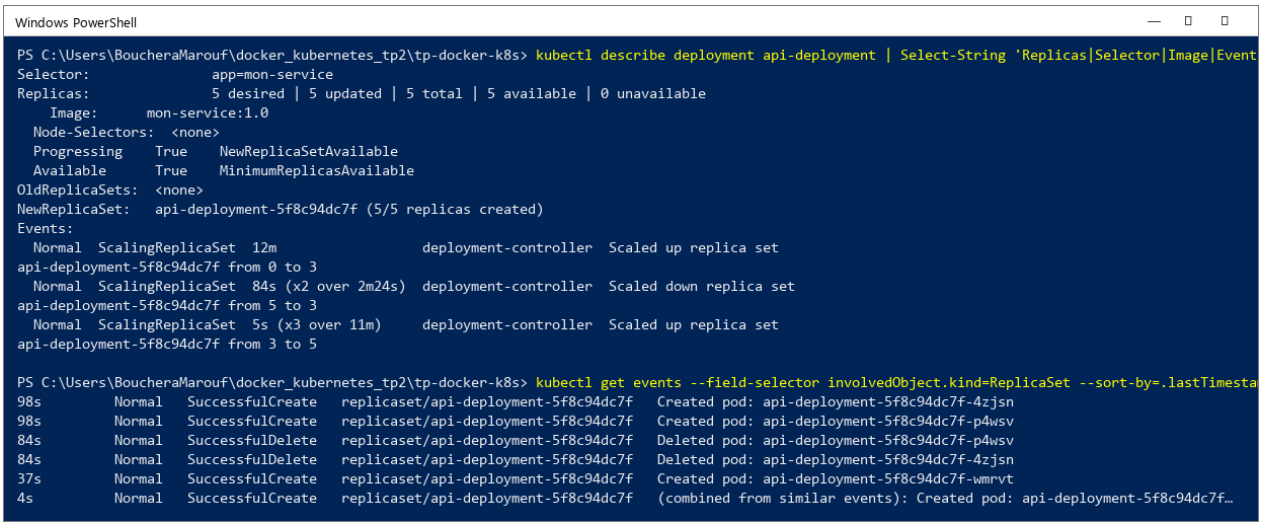


Figure 10 — description du Deployment et événements de création de pods

7.2 — Montée en charge (scaling)

La commande *kubectl scale deployment api-deployment --replicas=5* modifie l'état désiré. Kubernetes crée immédiatement deux pods supplémentaires, et le Deployment affiche **5/5 pods prêts** en quelques secondes. Le Service inclut automatiquement ces nouveaux pods dans la répartition de charge, puisqu'ils portent le même label.

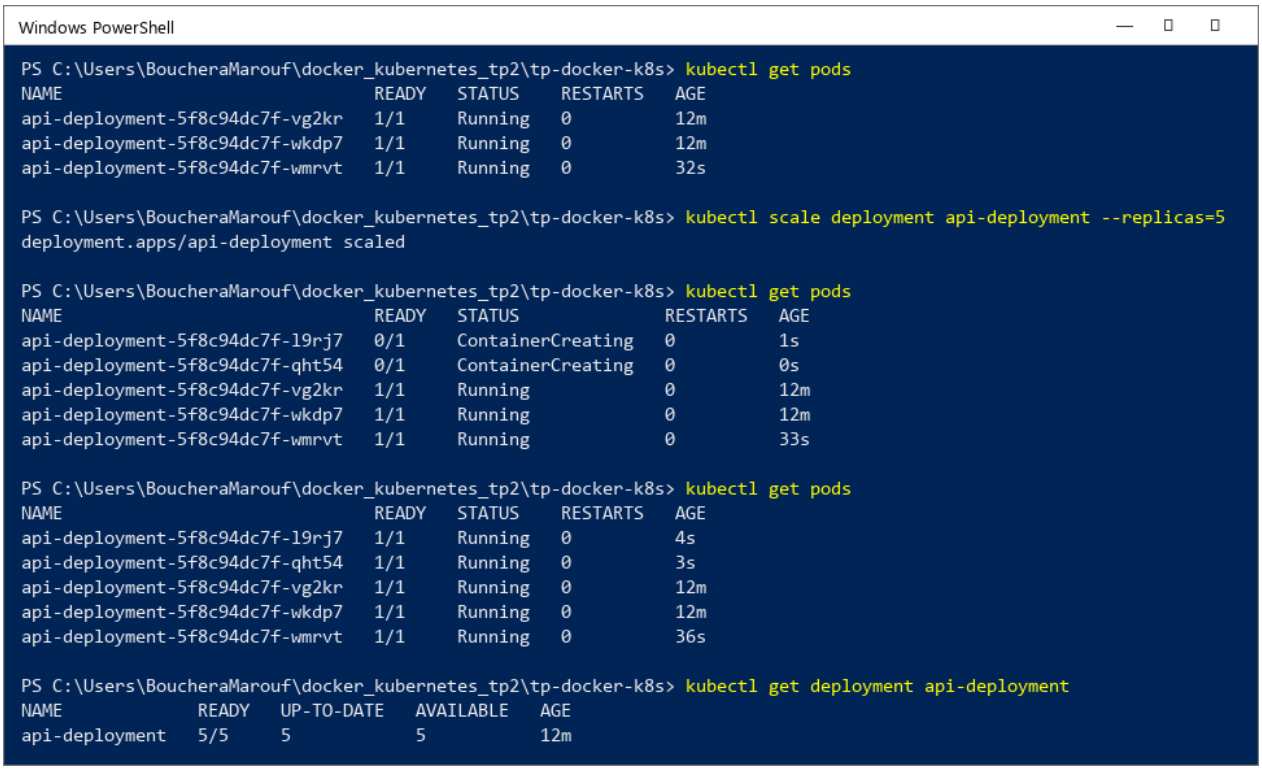


Figure 11 — passage de 3 à 5 réplicas

9. Versionnement du projet (Étape 8)

Le projet est versionné dans un dépôt Git. Le commit contient le Dockerfile, le fichier docker-compose.yml, les manifests Kubernetes (deployment.yaml et service.yaml), le code de l'API, les réponses et les captures.

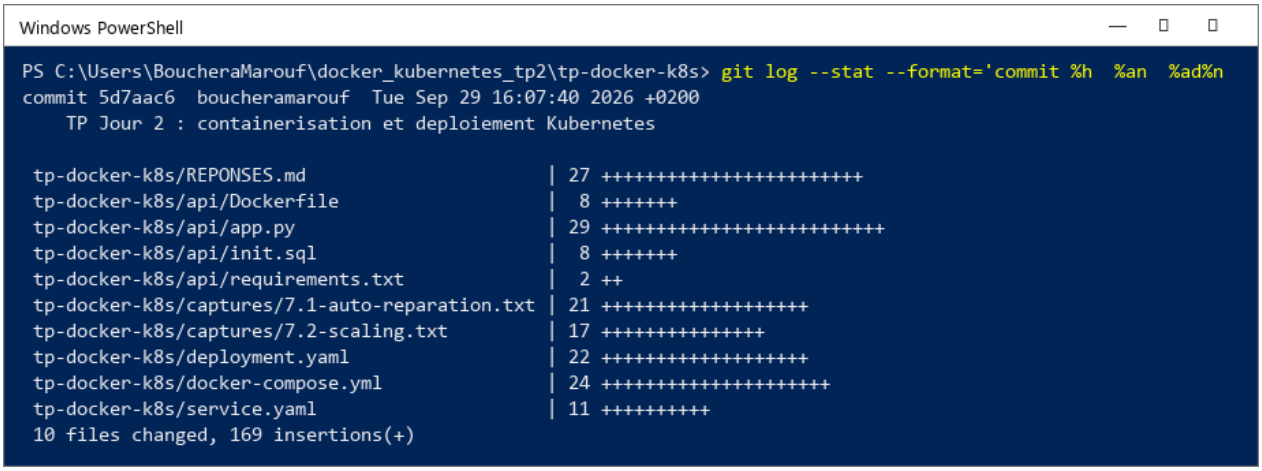


Figure 12 — historique Git du projet

10. Réponses aux questions de compréhension

1. Quelle est la différence entre une image Docker et un conteneur Docker ?

Une **image** est un modèle **immuable** en lecture seule, composé de couches empilées, qui contient tout le nécessaire pour exécuter l'application : système de fichiers, dépendances, code et commande de démarrage. Elle est construite une fois (*docker build*) et peut être partagée via un registre. Un **conteneur** est une **instance en cours d'exécution** d'une image : un processus isolé (namespaces, cgroups), auquel Docker ajoute une couche inscriptible éphémère. Une même image peut donner naissance à plusieurs conteneurs : dans ce TP, l'image *mon-service:1.0* a servi à lancer le conteneur Compose et les 3 puis 5 pods Kubernetes. L'analogie courante est celle d'une classe (l'image) et de ses objets (les conteneurs).

2. Pourquoi les données de la base sont-elles perdues si l'on supprime le volume, mais pas si l'on supprime seulement le conteneur ?

PostgreSQL écrit ses fichiers dans */var/lib/postgresql/data*. Dans notre configuration, ce répertoire est monté sur le **volume nommé donnees_db**, un espace de stockage géré par Docker **en dehors du cycle de vie du conteneur**. Supprimer le conteneur (*docker compose down*) ne détruit que sa couche inscriptible : le volume reste, et le nouveau conteneur s'y rattache et retrouve les données. Supprimer le volume (*docker compose down -v*) détruit en revanche le stockage lui-même : PostgreSQL repart d'un répertoire vide et réexécute *init.sql*. Cela a été vérifié à la section 5 avec la ligne « Test persistance ».

3. Un Deployment Kubernetes déclaré avec replicas: 3 perd un pod. Que fait Kubernetes, et pourquoi parle-t-on d'approche déclarative ?

Le contrôleur du Deployment, par l'intermédiaire de son **ReplicaSet**, observe que l'état réel (2 pods) diverge de l'état désiré (3 pods). Il **crée automatiquement un nouveau pod** à partir du template, que le scheduler place sur un nœud disponible. C'est l'auto-réparation observée à la section 8 : un nouveau pod est apparu en quelques secondes. On parle d'approche **déclarative** parce qu'on ne décrit pas les actions à effectuer (« démarre un conteneur », « relance-le s'il tombe »), mais l'**état final souhaité** dans un manifest YAML versionnable. Kubernetes exécute en continu une **boucle de réconciliation** qui compare l'état observé à l'état déclaré et agit pour les faire converger. À l'inverse, une approche impérative consisterait à enchaîner soi-même des commandes, comme avec *docker run*.

4. Quel est le rôle du Service par rapport aux pods, sachant que ces derniers sont recréés régulièrement avec une nouvelle adresse IP ?

Les pods sont **éphémères** : lors de l'auto-réparation ou du scaling, ils sont créés ou détruits et reçoivent à chaque fois une nouvelle adresse IP. On ne peut donc pas s'y adresser directement de façon fiable. Le **Service** fournit une **abstraction stable** : un nom DNS (*api-service*) et une IP virtuelle fixe (ClusterIP) qui ne changent pas. Il sélectionne dynamiquement les pods par leur label (*app: mon-service*), maintient à jour la liste de leurs adresses (les *endpoints*) et **répartit la charge** entre eux. Les nouveaux pods créés par le scaling ont ainsi été intégrés automatiquement. Avec le type **NodePort**, le Service est de plus accessible depuis l'extérieur du cluster.

5. Citez deux limites de Docker seul (sans Kubernetes).

- **Pas d'auto-réparation à l'échelle d'un cluster** : si un conteneur ou la machine hôte tombe, rien ne recrée automatiquement le service ailleurs. Une politique *restart* ne relance que sur le même hôte, et elle est inopérante si l'hôte lui-même est en panne.

- **Pas de mise à l'échelle ni de répartition de charge natives sur plusieurs machines** : Docker et Compose gèrent un seul hôte. Il n'existe pas d'équivalent simple à *kubectl scale --replicas=5* avec un point d'entrée unique qui répartit le trafic.
- Autres limites : pas de mises à jour progressives (*rolling update*) ni de retour arrière intégrés, pas de gestion centralisée des secrets et de la configuration, et pas de placement automatique des conteneurs selon les ressources disponibles (scheduling).

11. Difficultés rencontrées et remarques

- **Minikube non installé** : installé via *winget*. Le dossier d'installation n'est ajouté au PATH que dans les nouveaux terminaux : il faut fermer et rouvrir le terminal, comme l'indique l'énoncé.
- **Écart de versions kubectl / cluster** : le kubectl de Docker Desktop (v1.34) est plus ancien que le cluster Minikube (v1.37). Minikube affiche un avertissement, mais toutes les commandes du TP ont fonctionné. On peut l'éviter en installant kubectl via *winget install -e --id Kubernetes.kubectl* ou en utilisant *minikube kubectl* --.
- **Route /ventes indisponible dans Kubernetes** : la base étant restée sous Docker Compose, le nom *db* n'est pas résolu dans le cluster. Ce comportement est prévu par l'énoncé. La solution complète consiste à déployer PostgreSQL dans le cluster (Deployment, Service nommé *db* et PersistentVolumeClaim).
- **Suppression de pod lente (environ 30 s)** : *kubectl delete pod* attend la fin du délai de grâce. Le processus Python, lancé en PID 1, ne traite pas le signal SIGTERM, et Kubernetes l'arrête donc de force au bout de 30 s. Le pod de remplacement est toutefois créé immédiatement. L'option *--wait=false* a été utilisée pour les captures.
- **Accès au NodePort sous Windows** : avec le driver docker, l'IP du nœud n'est pas joignable directement. Il faut utiliser *kubectl port-forward* ou *minikube service api-service*, qui ouvre un tunnel.

Pistes d'amélioration (section « Pour aller plus loin »)

- Déployer PostgreSQL sur Kubernetes (Deployment, Service et PersistentVolumeClaim) pour obtenir une pile 100 % Kubernetes.
- Stocker les identifiants de la base dans un **Secret** Kubernetes, injecté via *valueFrom.secretKeyRef*, plutôt qu'en clair.
- Ajouter une **readinessProbe** et une **livenessProbe** HTTP sur */health* : le Service n'enverrait du trafic qu'aux pods prêts, et un pod bloqué serait redémarré automatiquement.
- Ajouter un healthcheck sur la base dans Compose, avec *depends_on: condition: service_healthy*, pour que l'API attende que PostgreSQL soit réellement prêt.

12. Conclusion

Ce TP a permis de parcourir toute la chaîne, du code à l'orchestration. L'API a été containerisée avec un Dockerfile optimisé pour le cache, puis assemblée avec sa base de données grâce à Docker Compose, en séparant clairement le cycle de vie des conteneurs de celui des données (volume persistant). Le déploiement sur Minikube a ensuite montré l'apport d'un orchestrateur : grâce à l'approche déclarative, Kubernetes a recréé seul un pod supprimé et a porté le service de 3 à 5 réplicas avec une seule commande. Le Service a assuré un point d'accès stable malgré le caractère éphémère des pods. Ces mécanismes sont la base d'un déploiement fiable et évolutif des solutions de données.